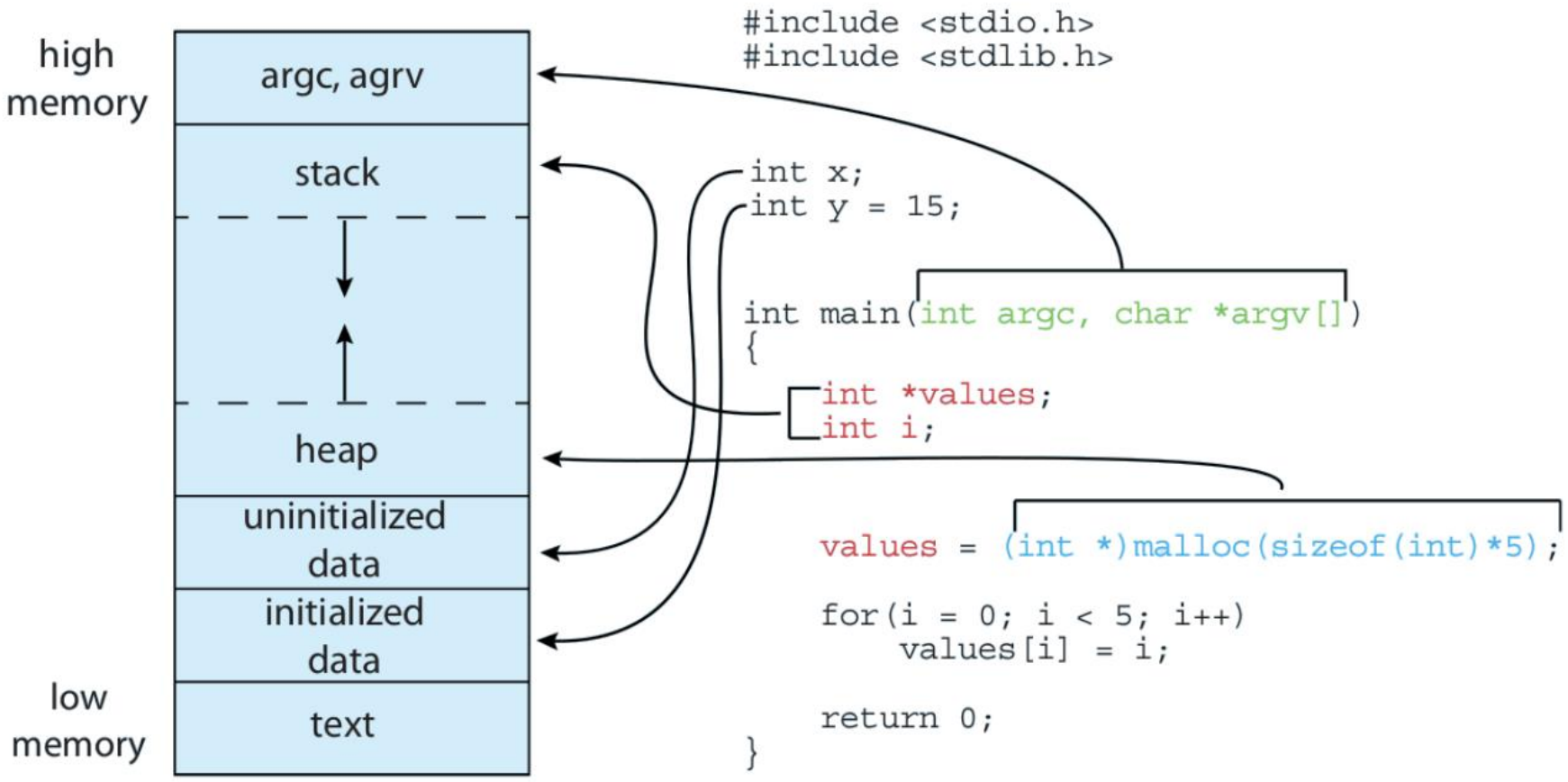
Recap: Process





Thread

Ivy Peng

Associate Professor of Computer Science

EECS, KTH



Outline

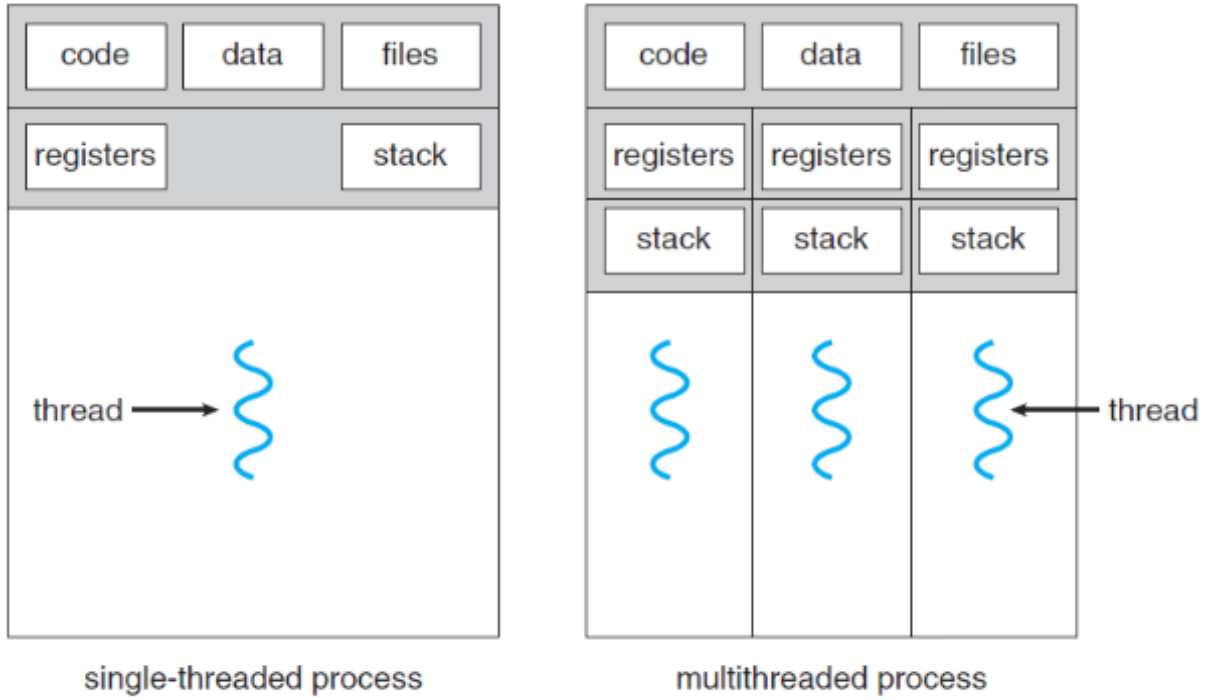
- **What is a thread**
- Multi-core Programming
- Pthreads
- Threading Issues

By the End of This Lecture

- Q1 Threads in one process: what do they share, and what does each own privately?
- Q2 Concurrency vs. parallelism: can you have concurrency without parallelism?
- Q3 Your machine has 8 cores. Why won't an 8-thread program run 8x faster?
- Q4 A 8-threaded program increments a shared value. Is the result value deterministic?

Processes vs. Threads

- A traditional process: has a single thread.
- Multiple threads in a process can perform multiple tasks at a time
  - Each thread is a basic unit of CPU utilization
- Threads in a process share code section, data section, and other OS resources, e.g., open files.
- Threads in a process have separate thread ids, program counters, register sets, and stacks



single-threaded process

multithreaded process

Threads and Multitasking

- Multiple tasks of an application can be implemented by separate threads.
- Update display
- Fetch data
- Spell checking
- Answer a network request



Limitations of Using Processes

- Process creation is time consuming and resource intensive
  - Load the executable into memory
- The resource overhead of new processes
  - Duplicate code, data, file sections ,use more memory
- Inter-process communication is slow

Benefits of Using Threads



Responsiveness: allow continued execution if part of the process is blocked.



Resource Sharing: threads share process resources, which is easier than shared memory or message passing.



Economy: thread switching has lower overhead than context switching.



Scalability: The process can take advantage of multiprocessor architectures



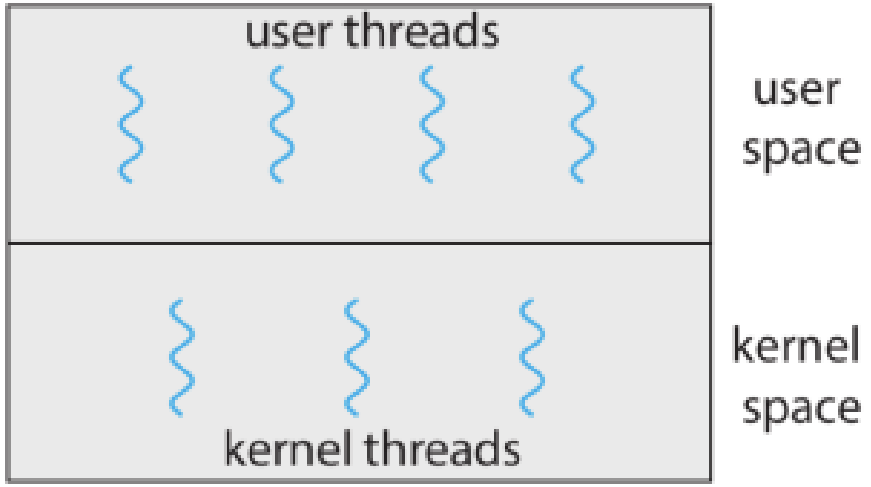
When comes to scheduling

- A thread is the unit of scheduling:
  - threads are scheduled to different CPUs on multi-core systems for performance, lower interference
- A process is the unit of resource ownership: memory, CPUs

User Threads and Kernel Threads

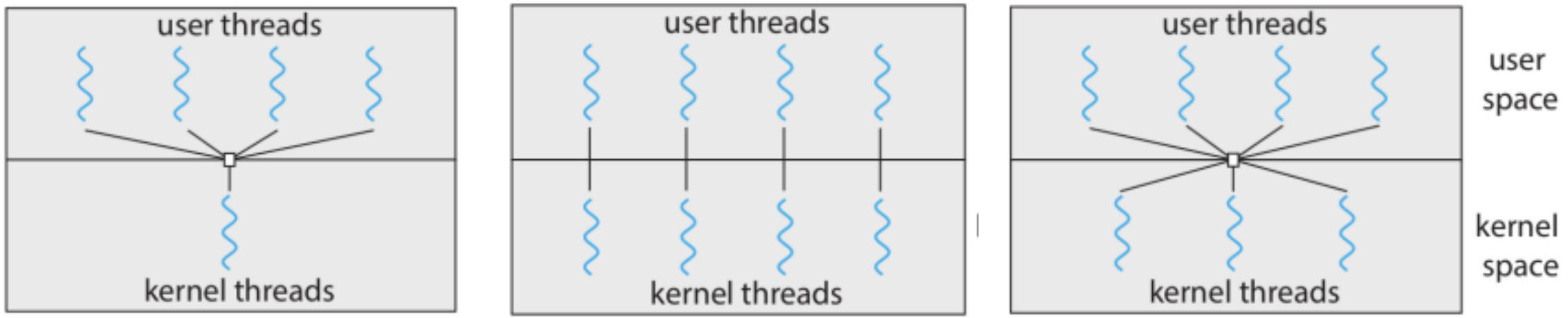
- When threads are managed at the user-level, i.e., **User Threads,** no kernel support is needed but managed by the user-level threads library. Three primary thread libraries include:
  - POSIX pthreads (the focus in this course)
  - Windows threads
  - Java threads

- Modern OS also uses threads in the kernel to improve efficiency, i.e., **Kernel threads**. They are managed by the OS directly.
  - Windows
  - Linux
  - MacOS



3 Multi-Threading Models

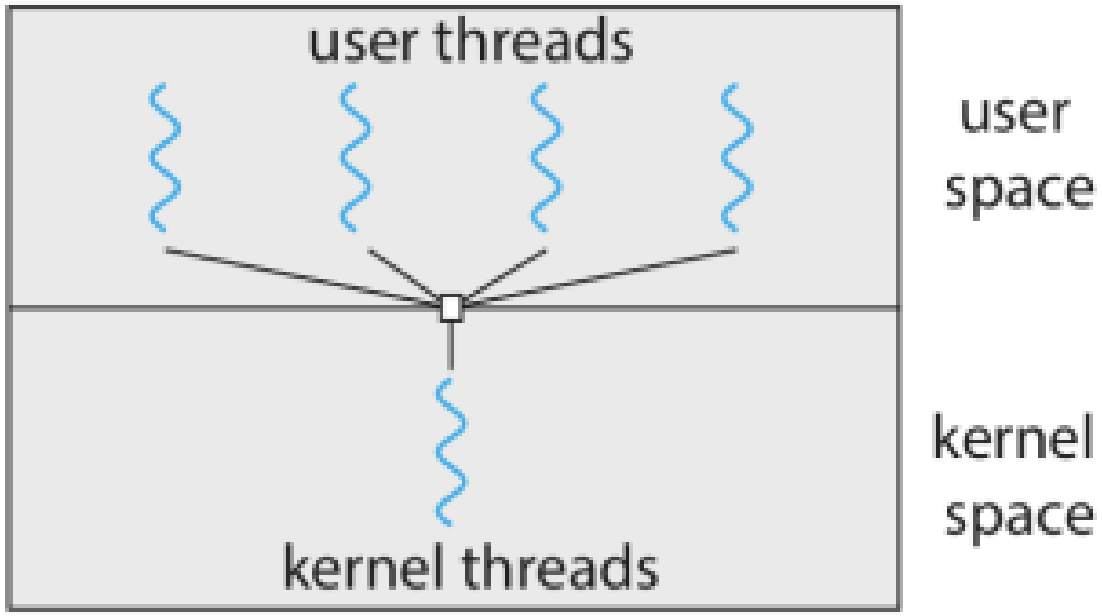
- User Threads need to be mapped to Kernel Threads
- Three mapping models between user and kernel threads:
  - Many-to-One
  - One-to-One
  - Many-to-Many



Many-to-One Model

- Many user-level threads are mapped to a single kernel thread.
- One thread blocking causes all to block., e.g., a long system call
- Multiple threads may not run in parallel on a multi-core system because only one thread may be in the kernel at a time.
- Very few systems currently use this model.
  - Solaris green threads
  - GNU portable threads

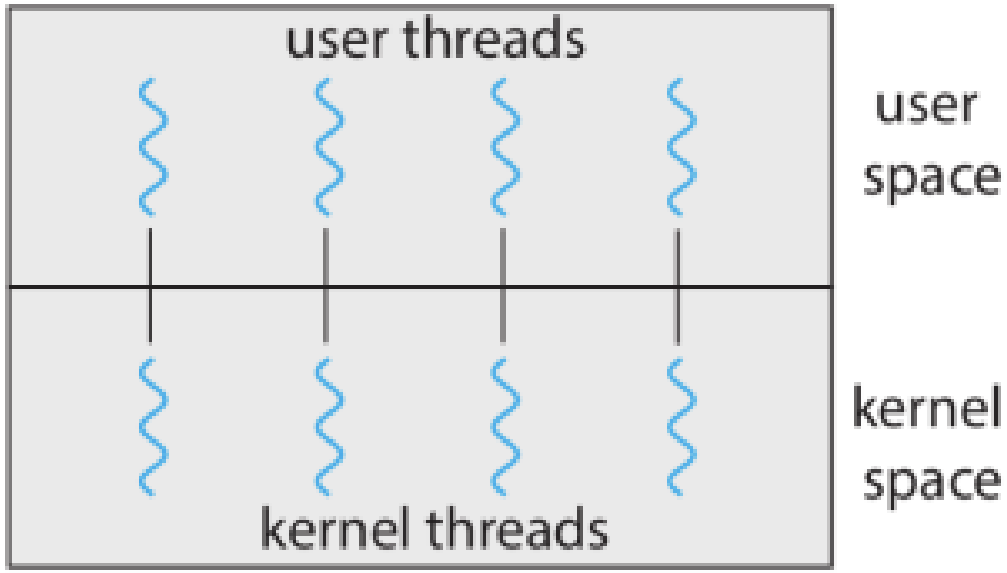




One-to-One Model

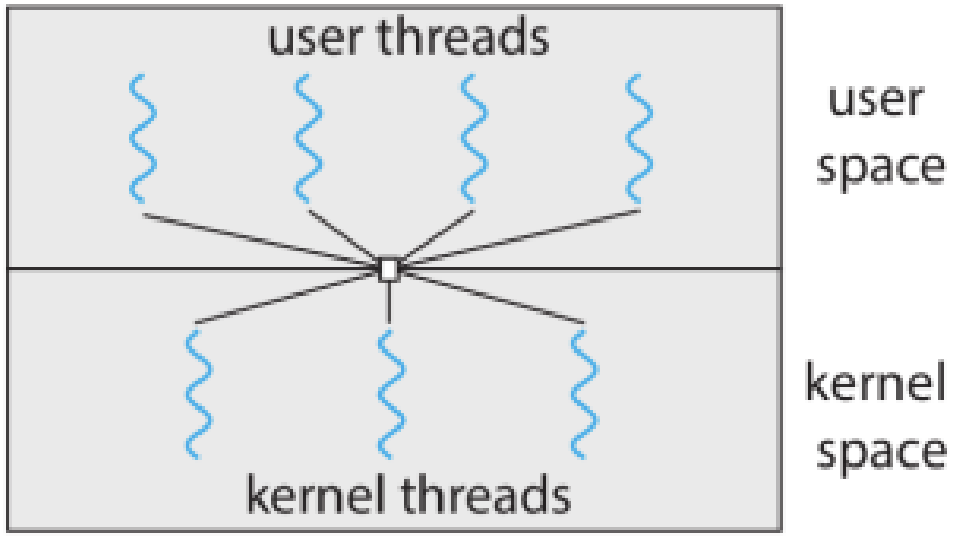
- Each user-level thread maps to one kernel thread.
- Creating a user-level thread creates a kernel thread.
- More concurrency than many-to-one.
- The number of threads per process is sometimes restricted d to overhead.
- Examples:
  - Windows
  - Linux





Many-to-Many Model

- This allows many user-level threads to be mapped to many kernel threads.
- This allows the OS to create a sufficient number of kernel threads
- Examples:
  - Windows with the ThreadFiber package
  - Otherwise, it is not very common





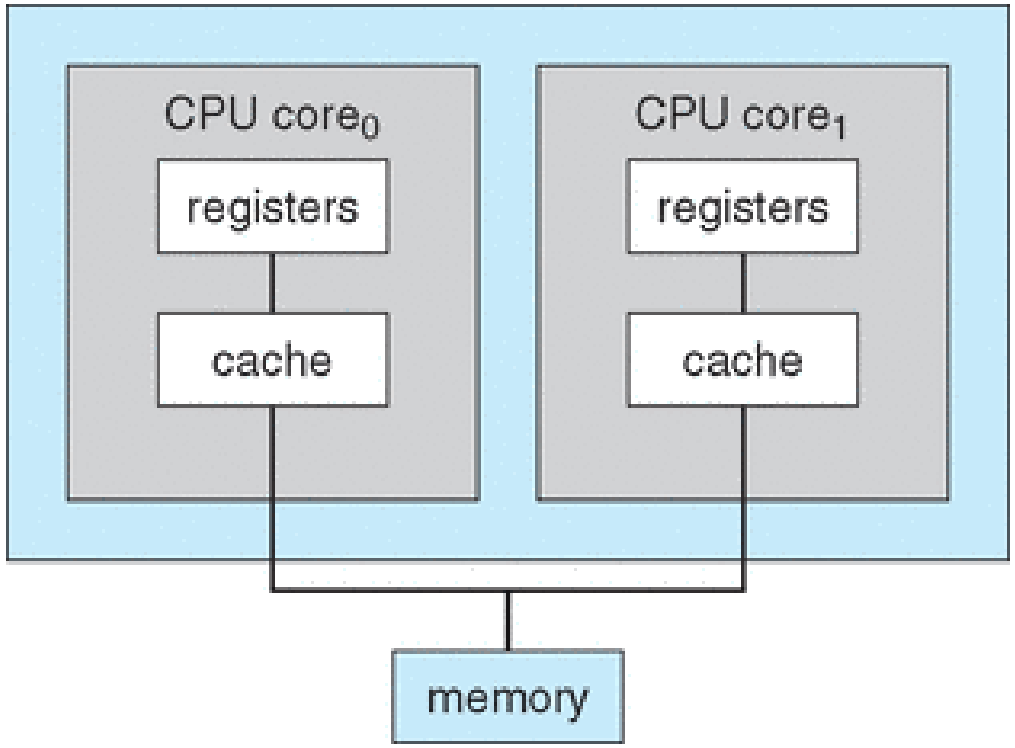


Outline

- What is a thread
- **Multi-core Programming**
- Pthreads
- Threading Issues

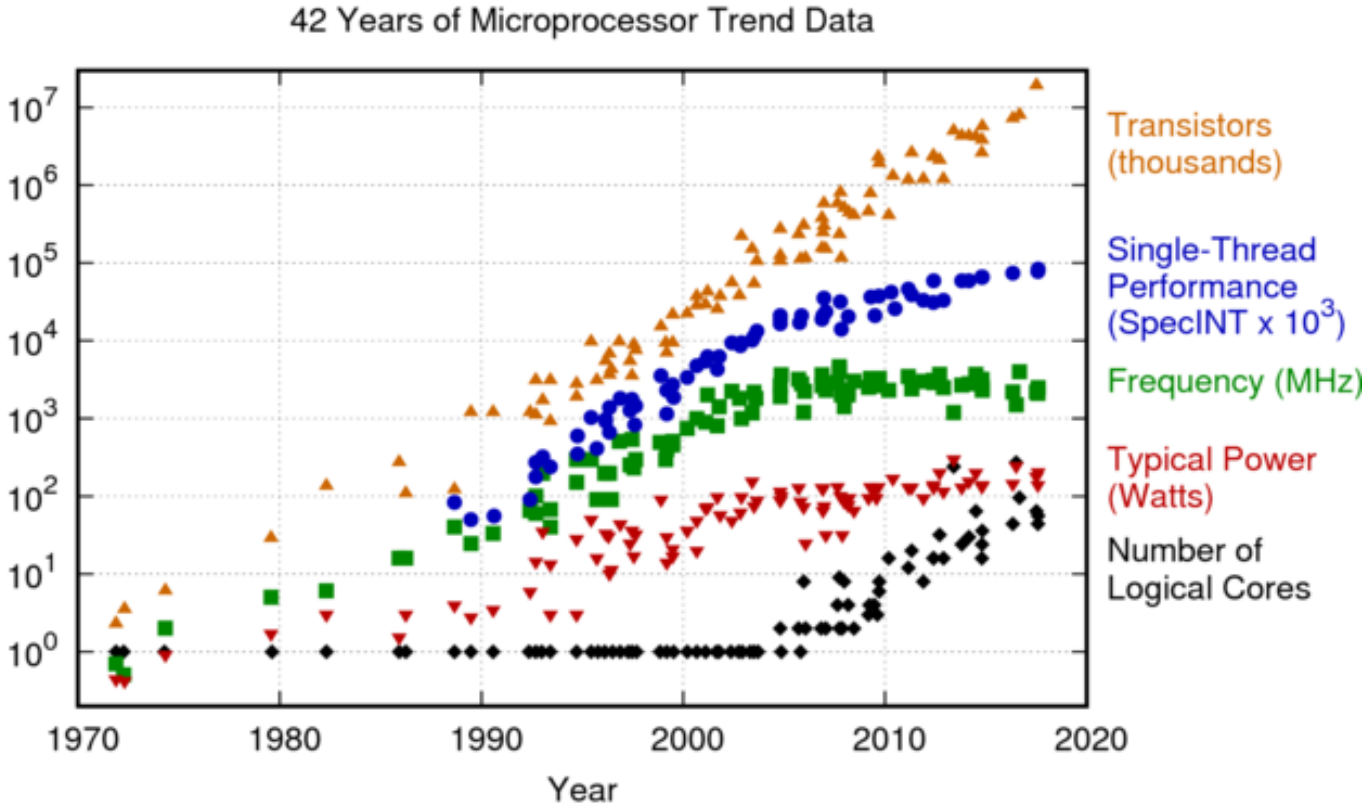
Multi-core Systems

- Users need more computing performance:
  - Instead of using single-CPU, use multiple CPUs
- A similar trend in system design: multi-core systems
  - Each core appears as a separate processor.
- Multi-threaded programming improves **concurrency**
  - Concurrency refers to have multiple different tasks ongoing, It could be achieved via time-sharing one CPU or multiple CPUs
- Multi-threaded programming makes more efficient use of multiple cores.



The trend of CPU Hardware

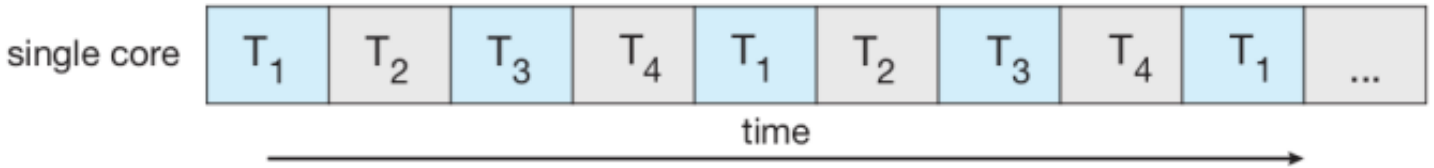
- Single thread performance and core frequency staggered mid-2010
  - Since Moore's Law ended at the same time: doubling the components on a semiconductor chip every two years has stopped
- However, the number of transistors and cores continue to grow exponentially



https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/

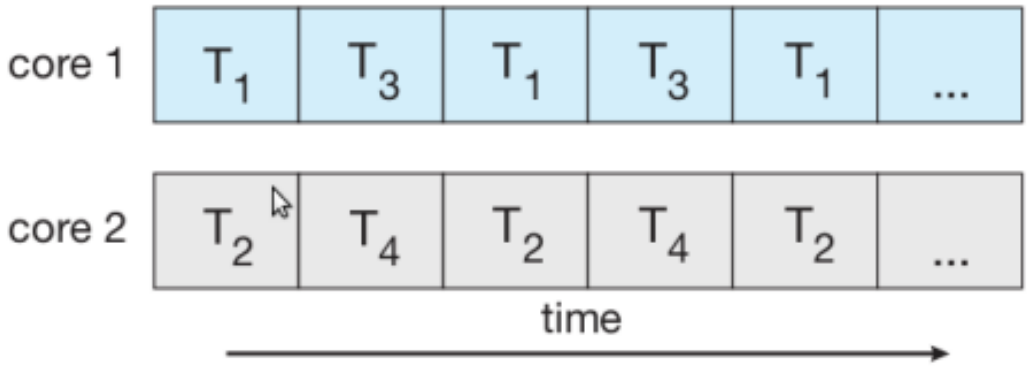
Concurrency vs. Parallelism

- Concurrency: supporting more than one task, allowing all the tasks to make progress (but not necessarily simultaneously).
  - A scheduler providing concurrency.
  - Concurrent execution is possible on a single-core system.



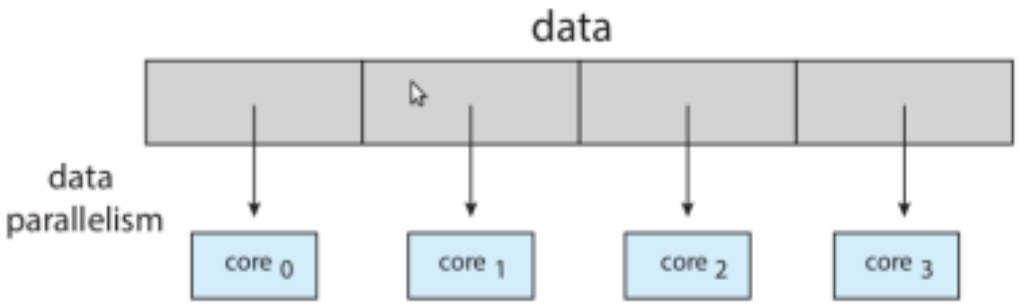
Concurrency vs. Parallelism

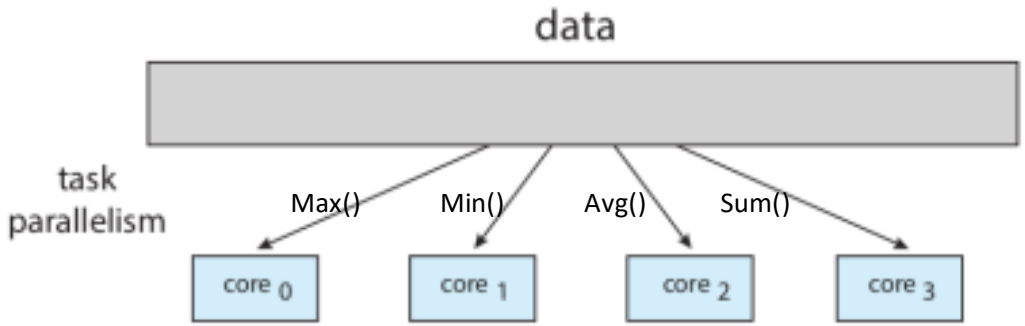
- Parallelism means performing more than one task simultaneously.
- Parallelism is achieved on multi-core systems
- Concurrency without parallelism? Possible, time-sharing one core.
- Parallelism without concurrency? Not Possible.



Data Parallel vs. Task Parallel

- Data parallelism distributes subsets of the same data across multiple cores, with the same operation on each.
- Task parallelism distributes threads across cores, each thread performing unique operation





Amdahl's Law: Why 8 Cores ≠ 8× Speedup

- A program has a serial fraction S (must run in order) and a parallel fraction 1 − S.
- Speedup on N cores $\leq 1 / (S + (1 - S) / N)$
  - as $N \to \infty$, speedup $\to 1/S$.
  - as $S \to 0$, speedup $\to N$.
- The serial fraction, not the core count, is the ceiling.

Serial fraction S	2 cores	4 cores	8 cores	∞ cores
5%	1.9×	3.5×	5.9×	**20×**
25%	1.6×	2.3×	2.9×	**4×**
50%	1.3×	1.6×	1.8×	**2×**

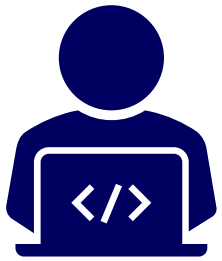


Outline

- What is a thread
- Multi-core Programming
- **Pthreads**
- Threading Issues

Thread Management

- Thread library provides programmers with API for creating and managing threads.
- Two primary ways of implementing this:
  - User-level libraries that entirely run in user space.
  - Kernel-level libraries that are supported by the OS.



Pthread

Either a user-level or a kernel-level library.



Windows thread

Kernel-level library.



Pthread

- A POSIX API for thread creation and synchronization.
- It is a specification, not implementation.
  - API specifies the behavior of the thread library; implementation is up to the development of the library.
  - Pthread is implemented in the standard libc in Linux
- Common in UNIX OSs, e.g., Solaris, Linux, Mac OS X

Optional: glibc pthread implementation

- https://sourceware.org/git/?p=glibc.git;a=tree;f=nptl

projects / glibc.git / tree

summary | shortlog | log | commit | commitdiff | tree
history | HEAD

malloc: Fix MALLOC_DEBUG

[glibc.git] / nptl /

drwxr-xr-x	..	
-rw-r--r--	DESIGN-systemtap-probes.txt	blob | history | raw
-rw-r--r--	Makefile	blob | history | raw
-rw-r--r--	TODO	blob | history | raw
-rw-r--r--	TODO-kernel	blob | history | raw
-rw-r--r--	TODO-testing	blob | history | raw
-rw-r--r--	pthread_create.c	blob | history | raw
-rw-r--r--	pthread_detach.c	blob | history | raw
-rw-r--r--	pthread_equal.c	blob | history | raw
-rw-r--r--	pthread_exit.c	blob | history | raw
-rw-r--r--	pthread_join.c	blob | history | raw
-rw-r--r--	pthread_join_common.c	blob | history | raw
-rw-r--r--	pthread_key_create.c	blob | history | raw
-rw-r--r--	pthread_key_delete.c	blob | history | raw
-rw-r--r--	pthread_keys.c	blob | history | raw
-rw-r--r--	pthread_kill.c	blob | history | raw

Thread ID

- The kernel schedules threads, and gives each a TID
  - similar to each process getting a PID
  - In a multithreaded process, every thread shares the PID but has a distinct TID.
  - Use gettid() to show the identity the OS assigns to the calling thread.

- On a Linux system, display the kernel TID by:
  - top -H
  - ps -L
  - /proc/<pid>/task/

Create a Thread

- pthread_create() defines and launches a new thread
- The new thread starts its execution from the thread_func
- Glibc exposes each thread entity in an opaque library handle called *pthread_t*

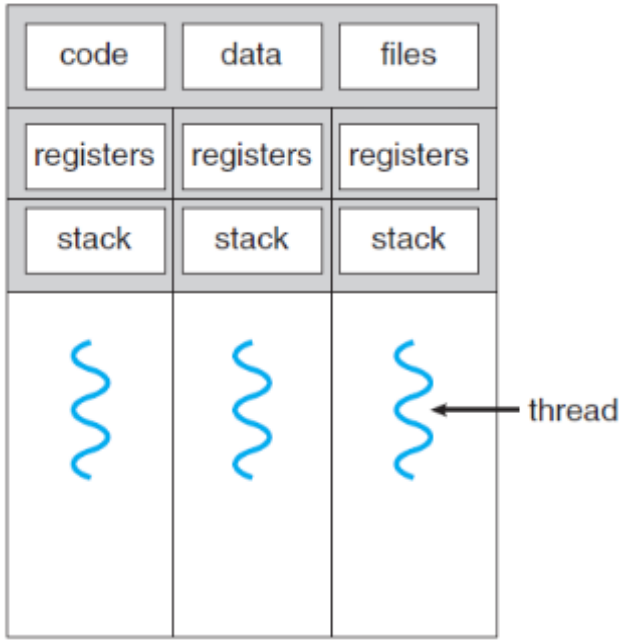
```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
  void *(*thread_func)(void *), void *arg);
```

```
void *thread_func(void *arg);
```

Threads in a Process

- A process can create multiple threads
- Threads in a process share code section, data section.
- 3-demo-pthread-shared.c, the printout at LINE A and B?



multithreaded process

```c
int value = 0;
void *thread_function(void *param) {
        value = 5;
        pthread_exit(0);
}

int main(int argc, char *argv[])
{
        pid_t pid;
        pthread_t tid;
        pthread_attr_t attr;
        pid = fork();
        if (pid == 0) { /* child process */
                pthread_attr_init(&attr);
                pthread_create(&tid,&attr,thread_function,NULL);
                pthread_join(tid,NULL);
                printf("CHILD: value = %d \n",value); /*LINE A*/
        }else if (pid > 0) { /* parent process */
                wait(NULL);
                printf("PARENT: value = %d \n",value); /*LINE B*/
        }
}
```

Terminating Threads

- A thread can terminate itself by calling pthread_exit().

```c
#include <pthread.h>

void pthread_exit(void *retval);
```

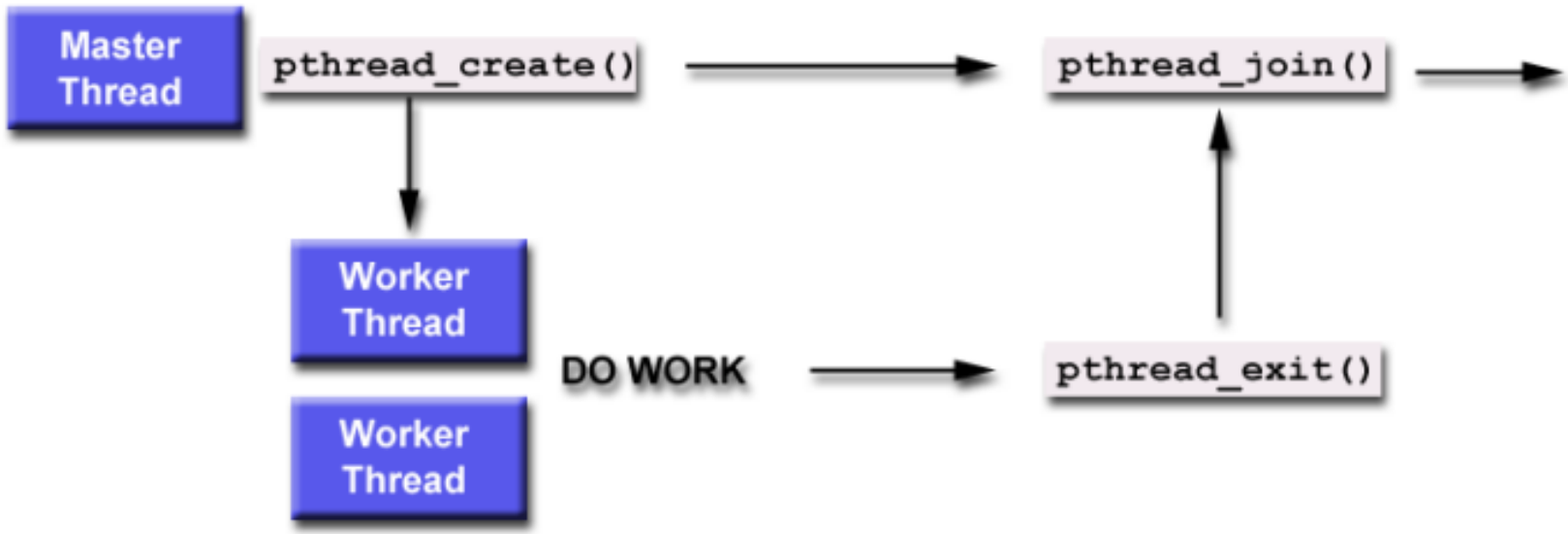
- A thread can also try to terminate other threads by calling pthread_cancel().

```c
#include <pthread.h>

int pthread_cancel(pthread_t thread);
```

Joining Threads

- if the master thread care about the value the child thread returns when it is done, use pthread_join()
- pthread_join() allows the master thread to block while waiting for the termination of another.



Joining Threads

- What is the printout?
  - nondeterministic order

```c
void *thread_func(void *message) {
  printf("%s\n", (const char *)message);
  return message;
}

int main(void) {
  pthread_t thread1, thread2;
  const char *message1 = "Thread 1";
  const char *message2 = "Thread 2";

  // Create two threads, each with a different message.
  pthread_create(&thread1, NULL, thread_func, (void *)message1);
  pthread_create(&thread2, NULL, thread_func, (void *)message2);

  // Wait for the threads to exit.
  pthread_join(thread1, NULL);
  pthread_join(thread2, NULL);

  return 0;
}
```

A pthread Example

- 3-demo-pthread-join.c
- Compile and link to the pthread library
- ~$ gcc test_pthread.c -pthread
- What is the printout?
- Is the result deterministic?

```
#define N 8
int value = 0;

void *thread_func(void *param) {
        value++;
        pthread_exit(0);
}

int main(int argc, char *argv[])
{
        pid_t pid;
        pthread_t tid;
        pthread_attr_t attr;

        pid = fork();
        if (pid == 0) {
                pthread_attr_init(&attr);
                for (int i = 0; i < N; i++) {
                        pthread_create(&tid, &attr, thread_func, NULL);
                        pthread_join(tid, NULL);
                }
                printf("CHILD: value = %d\n", value);
        }
        else if (pid > 0) {
                wait(NULL);
                printf("PARENT: value = %d\n", value);
        }
        else {
                perror("fork");
                return 1;
        }

        return 0;
}
```

A pthread Example: race

- In the previous slide, the result is deterministic because pthread_join() is called right after each pthread_create():
  - That is a sequential loop, (), the threads run one at a time, no concurrency.

- If thread join is moved at the end instead, and value++ becomes a race
  - The fix is mutual exclusion. That is the subject of the lecture on synchronization.

```
Thread A          Thread B          value
---------         ---------         -----
load value(0)                         0
                  load value(0)       0
add 1 -> 1                            0
                  add 1 -> 1          0
store 1                               1
                  store 1             1   <- two increments, +1 result
```

Two threads, two increments, final value 1, a lost update.

A data race: two threads access the same memory, unordered, and at least one writes.

Detaching Threads

- Use detach if the master thread do not need to know the value the child thread returns
- A detached thread will have its resource automatically released when terminated

```c
#include <pthread.h>

int pthread_join(pthread_t thread, void **retval);
int pthread_detach(pthread_t thread);
```



Resource Release

- When a thread terminates, its resource should be returned to the system to improve efficiency
  - E.g., A thread may have ‘malloc’ed large memory allocations
- The master thread should call either pthread_join() or pthread_detach() for each new thread
- Otherwise, their resource will wait till the parent process terminates to be released, instead of being released immediately when the thread terminates



Thread Cancellation

- Thread A can terminate thread B before thread B has finished.
- Here, Thread B is the target thread.
- Two general approaches:
  - Terminate the target thread immediately.
  - Allows the target thread to periodically check if it should be cancelled.
  - pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS|PTHREAD_CANCEL_DEFERRED)

Thread Cancellation

- 3-demo-pthread-cancel.c
- Assume thread 1 & 2 are scheduled to two separate CPUs and start running at the same time, at which timestamp will thread 2 exit?

```c
int main(int argc, char *argv[])
{
  pthread_t tid1, tid2;

  pthread_create(&tid1,NULL,thread_func1,NULL);
  pthread_create(&tid2,NULL,thread_func2,NULL);

  pthread_join(tid1,NULL);
  pthread_join(tid2,NULL);

  printf("Goodbye...\n");

}
```

```c
void *thread_func1(void *param) {
  int time_stamp = 0;

  while(1){
    printf("Thread 1: timestamp %d \n", time_stamp);
    sleep(4);
    time_stamp += 4;

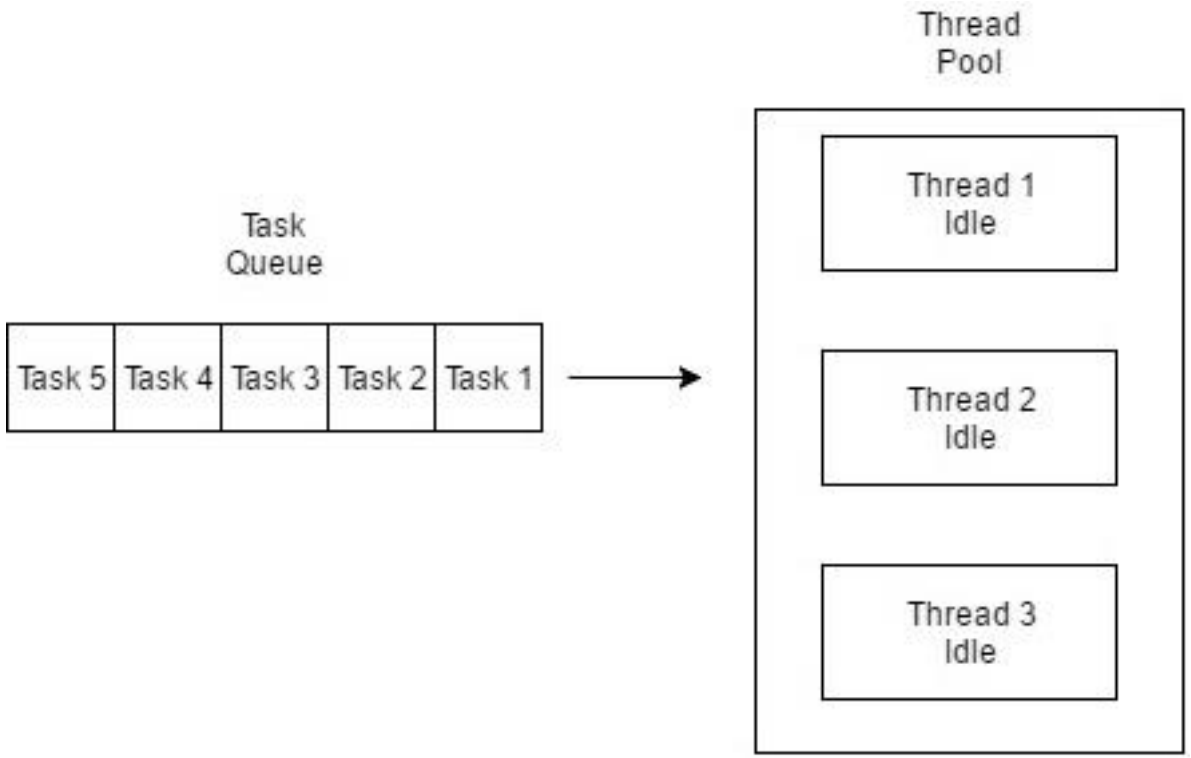
    if( time_stamp>4 ){
      pthread_cancel(target_thread);
      pthread_exit(NULL);
    }
  }
}
```

```c
void *thread_func2(void *param) {
  int time_stamp = 0;
  target_thread = pthread_self();

  while(1){
    printf("Thread 2: timestamp %d \n", time_stamp);
    sleep(1);
    time_stamp ++;
  }
}
```

Thread Pools

- It is slightly faster to service a request with an existing thread.
  - Create a number of threads in a pool where they await work.
  - This allows the number of threads in the application(s) to be bound to the pool size.





Outline

- What is a thread
- Multi-core Programming
- Pthreads
- **Threading Issues**



The fork() and exec() system calls

- Does fork() duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
  - POSIX: the child process is created with only one thread, i.e., the one calling fork()
- exec() usually works as normal: replace the entire process, including all threads.

Signal Handling

- What is signal?
  - A signal is generated by the occurrence of a particular event
  - The signal is delivered by the OS to a process
  - Once delivered, the signal must be handled
- A signal can be synchronous or asynchronous
  - Synchronous signals are delivered to the same process that performed the operation that caused the signal. e.g., : Invalid memory reference triggers SIGSEGV
  - Asynchronous signals are generated by an external event, .e.g., keyboard
- A signal handler is used to process signals

		value
SIGINT	Interrupt / press 'Ctrl-C'	2
SIGILL	Illegal Instruction	4
SIGKILL	Terminate immediately	9
SIGSEGV	Invalid memory reference	11

```
#include<signal.h>

int kill (pid_t pid , int sig); //generate a signal
```

Recap: Interrupt, Trap, Fault

A user program holds the CPU. How does the kernel ever get the CPU back?

	Interrupt	Trap	Fault
Example	Disk finished; timer fired; keyboard pressed	The syscall instruction	Page fault; divide-by-zero error
Timing	Asynchronous	Synchronous	Synchronous

A hardware path: event occurs (e.g., I/O event finishes) → CPU finishes the current instruction → index the interrupt vector table → switch to kernel mode → run the registered handler → restore state, return to user mode.

Example: the timer interrupt

Signal Handling

- Where should a signal be delivered in a multi-threaded process?
  - Deliver the signal to the thread to which the signal applies.
  - Deliver the signal to every thread in the process.
  - Deliver the signal to certain threads in the process.
  - Assign a specific thread to receive all signals for the process.

- A process may block certain signals: sigprocmask(). Similarly, a thread can block some signals: pthread_sigmask()

- POSIX pthread supports sending signals to specific threads:
  - int pthread_kill(pthread_t *thread*, int *sig*);

Thread-Local Storage (TLS)

- TLS is different from local variables:
  - Local variables are visible only during single-function invocation.
  - TLS is visible across function invocations.
- TLS allows each thread to have its own copy of data.
- Helpful when you do not have control over the thread creation process (i.e., thread pool)

- How to implement it?
- C and C++ use the keyword __thread in variable definition and declaration (ref: https://gcc.gnu.org/onlinedocs/gcc/Thread-Local.html)
- The compiler defines thread-local variables in .tdata and .tbss sections, which are created when a new thread is created

Back to the Four Opening Questions

- Q1 Threads in one process: what do they share, and what does each own privately?
  - Shared: code (text), global/heap data, open files, the PID.
  - Private: thread ID, program counter, register set, and stack.

- Q2 Concurrency vs. parallelism: can you have concurrency without parallelism?
  - Concurrency means several tasks make progress over an interval, not necessarily at the same time.
  - Parallelism means tasks run simultaneously, needing ≥2 cores.

- Q3 Your machine has 8 cores. Why won't an 8-thread program run 8x faster?
  - Amdahl's Law.

- Q4 A 8-threaded program increments a shared value. Is the result value deterministic?
  - Shared variables among threads
  - Multiple threads can update the shared variables unordered. Fix: synchronization (lectures later).